

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_87b22fa4-6ac\agent-  
a7ae0ab80736f8664.jsonl`  
- SHA-256: `8976e5ba282adda6b7c5057e04555fa42d7a2895a8d92ae5e4759b894803084e`  
- Source modified: `2026-06-21T10:47:36+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_87b22fa4-6ac`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T10:45:15.842Z)

Review the COMPUTE_DECOUPLED_SERVING **M6** change just committed on branch feat/serving-m6-cloudrun-dispatch (HEAD). Inspect with: git diff origin/master...HEAD , and read the full files:

```
- backend/compute/base.py (ComputeBackend ABC + ComputeJobSpec + ComputeResult)  
- backend/compute/cloud_run.py (CloudRunCompute: dispatch a Cloud Run Job via injectable  
CloudRunJobClient, then bucket-poll a done-marker via execution_policy.poll_until + the storage  
layer; GoogleCloudRunJobClient = real client, lazy google-cloud-run)  
- backend/compute/__init__.py (get_compute_backend(config) ? backend only for  
compute_target=cloud_run, else None = run inline)  
- backend/worker/__main__.py (cmd_infer: dispatch branch for cloud_run + _dispatch_remote;  
--done-uri + _write_done_marker on success)  
- deploy/cloudrun/Dockerfile + README.md (the L4 image)  
- tests/test_compute_backend.py  
CONTRACT: DEFAULT-OFF ? only compute_target=cloud_run dispatches; ""/local_gpu/cpu_mock run the  
worker's EXISTING in-process path BYTE-IDENTICALLY. The dispatched container must run INLINE  
(never re-dispatch) via forced compute_target=local_gpu+native. Completion = a bucket done-  
marker (not the Cloud Run API status), D16. The real client is owner-verified on M7 (CI uses a  
fake). Report ONLY real, grounded issues with file:line. Empty findings array if a lens finds  
nothing real ? do not invent nits.
```

LENS = DEFAULT-OFF SAFETY & RE-DISPATCH CORRECTNESS. (1) Trace cmd_infer: for compute_target in { "", local_gpu, cpu_mock } does get_compute_backend return None and the inline path run UNCHANGED? (2) The dispatched CONTAINER: CloudRunCompute appends container_overrides (compute_target=local_gpu + detector_impl=native) to the --set argv ? do these ACTUALLY win? Verify worker _apply_overrides applies --set in order (later wins) AND that the container's load_config + _apply_overrides yields compute_target=local_gpu so get_compute_backend returns None (no infinite re-dispatch). (3) Could a user --set or the cloud-serving config.json in the CONTAINER re-introduce compute_target=cloud_run and cause recursion? (4) The done-marker: confirm no --done-uri = no marker = unchanged. (5) Does the dispatch branch sit AFTER startup checks + _apply_overrides (so --set/profile is seen)? Find any default-OFF break or re-dispatch loop.

2. assistant (2026-06-21T10:45:19.146Z)

I'll review the M6 change on branch feat/serving-m6-cloudrun-dispatch with the DEFAULT-OFF safety & re-dispatch correctness lens. Let me start by examining the diff and the relevant files.

3. user (2026-06-21T10:45:22.876Z)

```
2266e91 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)  
---BRANCH---  
feat/serving-m6-cloudrun-dispatch
```

4. user (2026-06-21T10:45:23.081Z)

```
backend/compute/__init__.py | 61 ++++++++
backend/compute/base.py | 52 ++++++++
backend/compute/cloud_run.py | 141 ++++++++
backend/worker/__main__.py | 51 ++++++++
deploy/cloudrun/Dockerfile | 61 ++++++++
deploy/cloudrun/README.md | 59 ++++++++
tests/test_compute_backend.py | 222 ++++++++
7 files changed, 647 insertions(+)
```

5. assistant (2026-06-21T10:45:25.021Z)

Let me read the full files in parallel.

6. user (2026-06-21T10:45:26.006Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
5      DETECT?WINDOW?STITCH
6      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
7      (cloud-serving) ? without the core ever branching on the profile (D1).
8
9      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
10     non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
11     worker's
12     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
13     inline").
14     """
15     from __future__ import annotations
16
17     from abc import ABC, abstractmethod
18     from dataclasses import dataclass, field
19     from typing import Optional
20
21     @dataclass(frozen=True)
22     class ComputeJobSpec:
23         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
24         (re)compute
25         from the SAME input bytes, so the output always lands at
26         ``<out_uri>/outputs/<video_id>/`` ?
27         deterministic regardless of where the job ran."""
28
29         video_uri: str # input (gs://? for cloud; a path/local://
30         for local)
31         out_uri: str # output ROOT (gs://? bucket for cloud)
32         segmenter: Optional[str] = None
33         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
34         frozen/hashable.
35         config_overrides: tuple[str, ...] = ()
36
37     @dataclass(frozen=True)
38     class ComputeResult:
39         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
40         code
41         (0 ok / 2 validation-or-config / 3 segmenter-failure). ``video_id`` is known once
42         the job's
43         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
44         landed."""
45
46     38
```

```

39         returncode: int
40         outputs_uri: str = ""
41         video_id: str = ""
42         report: dict = field(default_factory=dict)
43
44
45     class ComputeBackend(ABC):
46         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
47         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
48         execution identically."""
49
50         @abstractmethod
51         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
52             """Run ``spec`` to completion and return its ``ComputeResult``."""
53

```

7. user (2026-06-21T10:45:26.461Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12            ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14        The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15        so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17        The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18        with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19        owner-verified on the M7 real run.
20        """
21
22        from __future__ import annotations
23
24        import json
25        import logging
26        import uuid
27        from abc import ABC, abstractmethod
28        from typing import Any, List, Optional
29
30        from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
31        from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
32        from backend.storage import fetch, parse_uri
33        from backend.storage import get_storage
34

```

```

35     LOG = logging.getLogger("compute.cloud_run")
36
37     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
38     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
39     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
40         "deployment.compute_target=local_gpu",
41         "indexing.detector_impl=native",
42     )
43
44
45     class CloudRunJobClient(ABC):
46         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
47
48         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
49         overrides the job's container ``args`` for this execution and starts it."""
50
51         @abstractmethod
52         def run_job(self, job_name: str, argv: List[str], *, region: str,
53                     project: Optional[str] = None) -> str:
54             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
55             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
56
57
58     class GoogleCloudRunJobClient(CloudRunJobClient):
59         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
60         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
61
62         def run_job(self, job_name: str, argv: List[str], *, region: str,
63                     project: Optional[str] = None) -> str:
64             try:
65                 from google.cloud import run_v2 # type: ignore[attr-defined]
66             except Exception as exc: # pragma: no cover - real SDK not present in CI
67                 raise RuntimeError(
68                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
69                     "control plane (it is intentionally NOT a CI/test dependency).")
70             ) from exc
71             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
72             name = client.job_path(project, region, job_name)
73             overrides = run_v2.RunJobRequest.Overrides(
74                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
75             )
76             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
77             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
78
79
80     class CloudRunCompute(ComputeBackend):
81         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
82
83         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
84                     project: Optional[str] = None, storage_config: Optional[dict] = None,
85                     policy: ExecutionPolicy = DEFAULT_POLICY,
86                     token: Optional[str] = None,

```

```

87         container_overrides: Optional[tuple[str, ...]] = None) -> None:
88     self._client = run_client
89     self._job_name = job_name
90     self._region = region
91     self._project = project
92     self._storage_config = storage_config or {}
93     self._policy = policy
94     # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
95     # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
96     self._token = token
97     self._container_overrides = (
98         _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
99     )
100
101     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
102         out_root = spec.out_uri.rstrip("/")
103         token = self._token or uuid.uuid4().hex
104         done_uri = f"{out_root}/_dispatch/{token}.done"
105         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
106                             "--done-uri", done_uri]
107         if spec.segmenter:
108             argv += ["--segmenter", spec.segmenter]
109         for kv in (*spec.config_overrides, *self._container_overrides):
110             argv += ["--set", kv]
111
112         execution = self._client.run_job(self._job_name, argv, region=self._region,
113                                         project=self._project)
114         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
115                self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
118                             label="cloud-run-infer", logger=LOG)
119         video_id = str(marker.get("video_id", ""))
120         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
121         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
122         rc = int(marker.get("returncode", 0))
123         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
124         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
125
126     # --- storage-backed completion signal (bucket-poll) ---
127     def _read_marker(self, done_uri: str) -> Optional[dict]:
128         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
129         ref = parse_uri(done_uri)
130         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
131             return None
132         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
133
134     def _read_json(self, uri: str) -> dict:
135         try:
136             local = fetch(uri, config=self._storage_config)
137             with open(local, "r", encoding="utf-8") as f:
138                 data: Any = json.load(f)
139             return data if isinstance(data, dict) else {}
140         except Exception:
141             return {}
142

```

8. user (2026-06-21T10:45:26.934Z)

```
1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
8      ``cpu_mock`` nothing
9      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
10     """
11
12     from __future__ import annotations
13
14     import os
15     from typing import Any, Optional
16
17     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20     __all__ = [
21         "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
22     ]
23
24     def _storage_dict(config: Any) -> dict:
25         storage = getattr(config, "storage", None)
26         if storage is None:
27             return {}
28         if hasattr(storage, "model_dump"):
29             return storage.model_dump() # type: ignore[no-any-return]
30         return dict(storage) if isinstance(storage, dict) else {}
31
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
44         ``CLOUD_RUN_REGION``
45         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
46         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
47         """
48         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
49         if target != "cloud_run":
50             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
51
52         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
53         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
```

```

54     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57     client = run_client if run_client is not None else GoogleCloudRunJobClient()
58     return CloudRunCompute(
59         run_client=client, job_name=job_name, region=region, project=project,
60         storage_config=_storage_dict(config), policy=policy, token=token,
61     )
62

```

9. user (2026-06-21T10:45:27.868Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs
11    are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13    key) ?
14    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17    CLI uses
18    (validator registry ? segmenter ? report), never a forked pipeline.
19    """
20    from __future__ import annotations
21
22    import argparse
23    import csv
24    import json
25    import logging
26    import os
27    import sys
28    import tempfile
29    from typing import Any, List
30
31    from backend import storage
32    from backend.compute import ComputeJobSpec, get_compute_backend
33    from backend.config import (
34        StartupCheckError,
35        enforce_startup_checks,
36        load_config,
37        probe_cuda_available,
38    )
39    from backend.infrastructure.database import Database
40    from backend.pipeline.segmenters.base import segmenter_registry
41    from backend.pipeline.validators.base import registry as validator_registry
42    from backend.reporting import emit_indexer_report
43    from backend.results import Failure
44    from backend.utils.hashing import compute_video_id
45
46    logger = logging.getLogger("backend.worker")
47
48    def _coerce(v: str) -> Any:
49        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50        str."""
51        low = v.lower()
52        if low in ("true", "false"):

```

```

49         return low == "true"
50     for cast in (int, float):
51         try:
52             return cast(v)
53         except ValueError:
54             pass
55     return v
56
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")
62             if not sep:
63                 raise ValueError(f"--set expects k=v, got {kv!r}")
64             parts = key.split(".")
65             obj = config
66             for p in parts[:-1]:
67                 obj = getattr(obj, p)
68             setattr(obj, parts[-1], _coerce(val))
69
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF

```



```

path a true
106     no-op of work, not just of output."""
107     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108     cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109     try:
110         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111         return True
112     except StartupCheckError:
113         return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch."""
120         spec = ComputeJobSpec(
121             video_uri=args.video_uri, out_uri=args.out_uri,
122             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
123         )
124         result = compute.run_infer(spec)
125         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
126                     result.video_id, result.returncode, result.outputs_uri)
127         return result.returncode
128
129
130     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
131                             status: str, storage_cfg: dict, scratch: str) -> None:
132         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
133         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
134         try:
135             marker = {"video_id": video_id, "returncode": returncode,
136                      "segment_count": segment_count, "status": status}
137             local = os.path.join(scratch, "_done.json")
138             with open(local, "w", encoding="utf-8") as f:
139                 json.dump(marker, f)
140             storage.put(local, done_uri, config=storage_cfg)
141             logger.info("[worker] wrote completion marker -> %s", done_uri)
142         except Exception as e: # never fail a good run because the marker push hiccuped
143             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
144
145
146     def cmd_infer(args: argparse.Namespace) -> int:
147         config = load_config(args.config) if args.config else load_config()
148         _apply_overrides(config, args.set)
149         if not _run_startup_checks(config):
150             return 2
151
152         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
153         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
154         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
155         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
156         compute = get_compute_backend(config)
157         if compute is not None:
158             return _dispatch_remote(compute, args)

```

```

159
160     storage_cfg = config.storage.model_dump()
161
162     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
163     os.makedirs(scratch, exist_ok=True)
164     config.output.output_dir = scratch
165
166     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
167     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
168     print(f"[worker] pulled {args.video_uri} -> {local_video}")
169
170     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
171     video_id = compute_video_id(local_video)
172
173     # 3. VALIDATE (same registry as the main CLI).
174     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
175     failures = [r for r in val_results if not r.passed]
176     db = Database(os.path.join(scratch, config.output.db_name))
177     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
178
179     segments: List[dict] = []
180     segmenter_actual = None
181     segmentation_ran = False
182     status = "failed"
183     try:
184         if failures:
185             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
186             return 2
187         seg_name = args.segmenter or config.indexing.default_segmenter
188         segmenter_cls = segmenter_registry.get(seg_name)
189         if not segmenter_cls:
190             print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
191             return 2
192         segmenter_actual = seg_name
193         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
194         segmentation_ran = True
195         if isinstance(result, Failure):
196             print(f"[worker] segmenter FAILED: {result.message}")
197             return 3
198         segments = result.value if hasattr(result, "value") else result
199         status = "success"
200         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
201         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
202         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
203         if not segments:
204             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
205             logger.warning(
206                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
207                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
208                 "logs above; common causes: native runner UNAVAILABLE

```

```

(weights/repo/WASB_PYTHON env), "
213         "the WASB env produced no detections, or the input resolution differs
from the tuned "
214         "proxy resolution. Re-run with -v for the native command + frame
counts.",
215         video_id, segmenter_actual, detector_impl,
216     )
217     finally:
218         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
219         emit_indexer_report(
220             db=db, video_id=video_id, video_path=local_video, config=config,
221             val_results=val_results, val_context=val_context,
222             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
223             was_reingest=False, segmentation_ran=segmentation_ran,
224         )
225
226     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
227     out_local = os.path.join(scratch, "outputs", video_id)
228     os.makedirs(out_local, exist_ok=True)
229     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
230     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
231
232     out_prefix = args.out_uri.rstrip("/")
233     pushed = []
234     for fn in sorted(os.listdir(out_local)):
235         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
236         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
237         pushed.append(uri)
238
239     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
240     for u in pushed:
241         print("    ", u)
242
243     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
244     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
245     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
246     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
247     if getattr(args, "done_uri", None):
248         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
249     return 0
250
251
252     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
253     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
254     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
255
256
257     def _probe_video_fps_width(path: str):
258         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
259
260         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
261         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
262         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for

```

```

the
263         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
264         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
265         rather than guesses.
266         """
267         import json
268         import subprocess
269         try:
270             out = subprocess.check_output(
271                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
272                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
273                 stderr=subprocess.DEVNULL).decode()
274             st = (json.loads(out).get("streams") or [{}])[0]
275             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
276             fps = float(num) / float(den or "1")
277             width = float(st.get("width") or 0)
278             if fps > 0 and width > 0:
279                 return fps, width
280         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
281             pass
282         return None
283
284
285     def _resolve_train_detector(args: argparse.Namespace, config: Any):
286         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
287         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
288         stub=CPU mock). Returns the runner, or prints an error + returns None.
289
290         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
291         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
292         no shuttle detector and is rejected.
293         """
294         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
295
296         seg_cls = segmenter_registry.get(args.segmenter)
297         if not seg_cls:
298             print(f"[worker] unknown segmenter: {args.segmenter!r} "
299                   f"(have {list(segmenter_registry.list_available())})")
300             return None
301         seg = seg_cls(None, config)
302         if not isinstance(seg, TrajectoryHybridSegmenter):
303             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
304                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
305             return None
306         try:
307             runner, _ = seg._resolve_runner(config.get("indexing", {}))
308         except NotImplementedError as e:
309             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
310             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
311             print(f"[worker] detector not available: {e}")
312             return None
313         ok, msg = runner.healthcheck()
314         if not ok:
315             print(f"[worker] detector environment not ready: {msg}")
316             return None
317         print(f"[worker] detector ready ({args.segmenter}): {msg}")
318         return runner

```

```

319
320
321     def cmd_train(args: argparse.Namespace) -> int:
322         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
323         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
324
325         Two trajectory sources (the train pipeline + harness are identical either way):
326         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
327             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
328         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
329         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
330             is the M3.5 "first real training" path; only the trajectory source flips.
331         """
332         import subprocess
333
334         config = load_config(args.config) if args.config else load_config()
335         _apply_overrides(config, args.set)
336         if not _run_startup_checks(config):
337             return 2
338         storage_cfg = config.storage.model_dump()
339
340         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
341         labels_dir = os.path.join(scratch, "labels")
342         traj_dir = os.path.join(scratch, "trajectories")
343         videos_dir = os.path.join(scratch, "videos")
344         os.makedirs(labels_dir, exist_ok=True)
345         os.makedirs(traj_dir, exist_ok=True)
346
347         corpus = args.corpus_uri.rstrip("/")
348         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
349                             if u.lower().endswith(".csv"))
350         if len(label_uris) < 2:
351             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
352                   f"under {corpus}/labels/")
353             return 2
354
355         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
356         runner = None
357         video_by_stem: dict = {}
358         if args.trajectory_source == "detector":
359             os.makedirs(videos_dir, exist_ok=True)
360             runner = _resolve_train_detector(args, config)
361             if runner is None:
362                 return 2
363             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
364                 vname = storage.parse_uri(vuri).name
365                 if not vname.lower().endswith(_VIDEO_EXTS):
366                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
367                 video_by_stem[os.path.splitext(vname)[0]] = vuri
368
369         print(f"[worker] trajectory source: {args.trajectory_source}")
370         specs: List[dict] = []
371         for uri in label_uris:
372             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
373             name = fname.replace(".rallies.csv", "").replace(".csv", "")

```

```

374         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
375         if args.trajectory_source == "detector":
376             vuri = video_by_stem.get(name)
377             if not vuri:
378                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
379                 continue
380             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
381                                     config=storage_cfg)
382             video_id = compute_video_id(local_video)
383             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
384             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
385             meta = _probe_video_fps_width(local_video)
386             if meta is None:
387                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
388                 continue
389             fps, frame_width = meta
390             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
391             if not traj:
392                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
393                 continue
394             specs.append({"name": name, "trajectory": traj, "labels": local_label,
395                          "fps": fps, "frame_width": frame_width})
396         else:
397             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
398             traj = os.path.join(traj_dir, f"{name}_traj.csv")
399             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
400             specs.append({"name": name, "trajectory": traj, "labels": local_label,
401                          "fps": args.fps, "frame_width": args.frame_width})
402
403         if len(specs) < 2:
404             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
405             return 2
406         manifest_path = os.path.join(scratch, "manifest.json")
407         with open(manifest_path, "w", encoding="utf-8") as f:
408             json.dump(specs, f, indent=2)
409         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
410         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
411
412         out_dir = os.path.join(scratch, "artifacts")
413         cmd = [sys.executable, "-m", "training.gen0.harness",
414               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
415         if args.floor:
416             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
417                           if "://" in args.floor else args.floor)
418             cmd += ["--floor", floor_local]
419         print("[worker] training:", " ".join(cmd))
420         rc = subprocess.run(cmd).returncode
421         if rc != 0:
422             print(f"[worker] gen0 harness failed (rc={rc})")
423             return rc
424
425         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
426         bundle = os.path.join(out_dir, args.version)

```

```

427     out_prefix = args.out_uri.rstrip("/")
428     pushed = []
429     for fn in sorted(os.listdir(bundle)):
430         uri = f"{out_prefix}/weights/{args.version}/{fn}"
431         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
432         pushed.append(uri)
433     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
434     for u in pushed:
435         print("    ", u)
436     return 0
437
438
439     def build_parser() -> argparse.ArgumentParser:
440         p = argparse.ArgumentParser(prog="python -m backend.worker",
441                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
442         p.add_argument("-v", "--verbose", action="store_true",
443                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
444         sub = p.add_subparsers(dest="cmd", required=True)
445
446         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
447         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
448         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
449         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
450         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
451         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
452         pi.add_argument("--max-frames", type=int, default=None)
453         pi.add_argument("--done-uri", default=None,
454                       help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
455                       "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
456                       "Default-OFF: omit it for the normal local run.")
457         pi.add_argument("--set", action="append", default=[], metavar="k=v",
458                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
459         pi.set_defaults(func=cmd_infer)
460
461         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
462         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
463                       "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
464         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
465         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
466         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
467                       default="synth",
468                       help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
469                       "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
470                       "(needs videos/ in the corpus; honours detector_impl).")
471         pt.add_argument("--segmenter", default="wasb_hybrid",
472                       help="trajectory hybrid whose detector is used for --trajectory-
source detector")
473         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
474         pt.add_argument("--config", default=None)
475         pt.add_argument("--scratch", default=None)

```

```

475     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
476     pt.add_argument("--frame-width", type=float, default=1280.0,
477                     help="synth trajectory frame width (detector probes the video)")
478     pt.add_argument("--set", action="append", default=[], metavar="k=v")
479     pt.set_defaults(func=cmd_train)
480     return p
481
482
483     def _setup_logging(verbose: bool) -> None:
484         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
485         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
486         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
487         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
488         entry), not at import, so importing the module never reconfigures a caller's
logging."""
489         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
490         # configured logging at import), which would leave the level at WARNING and re-
suppress the
491         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
492         logging.basicConfig(
493             level=logging.DEBUG if verbose else logging.INFO,
494             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
495             datefmt="%H:%M:%S",
496             force=True,
497         )
498
499
500     def main(argv=None) -> int:
501         args = build_parser().parse_args(argv)
502         _setup_logging(getattr(args, "verbose", False))
503         return args.func(args)
504
505
506     if __name__ == "__main__":
507         sys.exit(main())
508

```

10. assistant (2026-06-21T10:45:30.364Z)

Now let me read the test file and the deploy files, plus inspect the supporting pieces (`\_apply\_overrides` ordering, `load\_config`, and the deployment config schema).

11. user (2026-06-21T10:45:31.321Z)

```

1     """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3     All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4     worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5     ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6     selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7     """
8     import json
9     import os
10

```



```

11     from backend.compute import ComputeJobSpec, get_compute_backend
12     from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
13     from backend.config import AppConfig
14     from backend.infrastructure.execution_policy import ExecutionPolicy
15
16     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
17     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
18
19
20     class _FakeRunClient(CloudRunJobClient):
21         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
22         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
23
24         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
25             self.calls: list[dict] = []
26             self.video_id, self.segment_count, self.returncode = video_id, segment_count,
returncode
27
28         def run_job(self, job_name, argv, *, region, project=None):
29             self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
30             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
31             done_uri = argv[argv.index("--done-uri") + 1]
32             outputs = os.path.join(out_uri, "outputs", self.video_id)
33             os.makedirs(outputs, exist_ok=True)
34             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
35                 json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
36                     "status": "success"}, f)
37             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
38             with open(done_uri, "w", encoding="utf-8") as f:
39                 json.dump({"video_id": self.video_id, "returncode": self.returncode,
40                     "segment_count": self.segment_count, "status": "success"}, f)
41             return "exec-123"
42
43
44     # ----- factory (default-
OFF)
45
46     def _cfg(compute_target="", storage_backend="local"):
47         return AppConfig.model_validate({
48             "deployment": {"compute_target": compute_target},
49             "storage": {"backend": storage_backend},
50         })
51
52
53     def test_factory_is_none_for_local_targets():
54         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
55         assert get_compute_backend(_cfg("")) is None
56         assert get_compute_backend(_cfg("local_gpu")) is None
57         assert get_compute_backend(_cfg("cpu_mock")) is None
58
59
60     def test_factory_returns_cloud_run_for_cloud_target():
61         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
62         assert isinstance(backend, CloudRunCompute)
63
64
65     # ----- CloudRunCompute dispatch
+ poll
66
67     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):

```

```

68         client = _FakeRunClient(video_id="abc123", segment_count=3)
69         backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-
south1",
70                                     project="proj", storage_config={}, policy=_FAST,
token="tok")
71         out = str(tmp_path / "bucket")
72         result = backend.run_infer(ComputeJobSpec(
73             video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))
74
75         assert result.returncode == 0
76         assert result.video_id == "abc123"
77         assert result.outputs_uri == f"{out}/outputs/abc123/"
78         assert result.report.get("segment_count") == 3
79         # exactly one Cloud Run Job execution was triggered, in the right region/project
80         assert len(client.calls) == 1
81         assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] ==
"asia-south1"
82
83
84     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
85         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
86         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
87         client = _FakeRunClient()
88         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST,
token="tok")
89         backend.run_infer(ComputeJobSpec(
90             video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
91             config_overrides={"sport=tennis"},))
92
93         argv = client.calls[0]["argv"]
94         assert argv[0] == "infer"
95         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
96         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
97         # the container-inline overrides come last (win) and the user override is carried
through
98         joined = " ".join(argv)
99         assert "deployment.compute_target=local_gpu" in joined
100        assert "indexing.detector_impl=native" in joined
101        assert "sport=tennis" in joined
102        # user override precedes the inline-forcing overrides in --set order
103        assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")
104
105
106    def test_unique_token_per_call_when_not_injected(tmp_path):
107        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
108        client = _FakeRunClient()
109        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
110        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk"))))
111        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk"))))
112        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
113        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
114        assert done1 != done2
115
116
117    # ----- worker wiring: dispatch branch +
done-marker
118
119    def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
120        """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-

```

```

process run)."""
121     from backend.compute.base import ComputeResult
122     from backend.worker import __main__ as wmain
123
124     seen = {}
125
126     class _FakeBackend:
127         def run_infer(self, spec):
128             seen["spec"] = spec
129             return ComputeResult(returncode=0, video_id="vidX",
outputs_uri="gs://b/outputs/vidX/")
130
131     monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
132     cfg = tmp_path / "c.json"
133     cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gs"}}}')
134     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
135         "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
136         "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
137     assert rc == 0
138     assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
139     assert seen["spec"].segmenter == "wasb_hybrid"
140
141
142     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
143         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
144         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
145         from backend.worker import __main__ as wmain
146
147         class _OK:
148             passed = True
149             validator_name = "fake"
150             message = "ok"
151
152             def model_dump(self):
153                 return {"validator_name": "fake", "passed": True, "message": "ok"}
154
155         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
lambda path, cfg: ([_OK()], {}))
156         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
157
158         monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
159                             lambda p: 30.0)
160
161         cfg = tmp_path / "c.json"
162         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
163                                     "output": {"output_dir": str(tmp_path / "o")}}))
164         video = tmp_path / "in.mp4"
165         video.write_bytes(b"FAKEVIDEO")
166         out = tmp_path / "bucket"
167         done = str(out / "_dispatch" / "tok.done")
168
169         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
170             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
171             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
done]))
172         assert rc == 0
173         marker = json.loads(open(done, encoding="utf-8").read())
174         assert marker["returncode"] == 0 and marker["status"] == "success"
175         assert marker["video_id"] and marker["segment_count"] >= 1
176
177

```

```

178     def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
179         """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
180         from backend.worker import __main__ as wmain
181
182         class _OK:
183             passed = True
184             validator_name = "fake"
185             message = "ok"
186
187             def model_dump(self):
188                 return {"validator_name": "fake", "passed": True, "message": "ok"}
189
190         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
191                             lambda path, cfg: ([_OK()], {}))
192         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
193
194         monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
195                             lambda p: 30.0)
196         cfg = tmp_path / "c.json"
197         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
198                                     "output": {"output_dir": str(tmp_path / "o")}}}))
199         video = tmp_path / "in.mp4"
200         video.write_bytes(b"FAKEVIDEO")
201         out = tmp_path / "bucket"
202         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
203             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
204             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
205         assert rc == 0
206         assert not (out / "_dispatch").exists() # no marker dir when --done-uri is omitted
207
208     # ----- Dockerfile structural guard
(no build)
209
210     def test_dockerfile_has_required_directives():
211         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
212         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
worker entrypoint."""
213         from pathlib import Path
214
215         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
216         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
217         assert "ffmpeg" in text # stitch/decode
218         assert "fonts-dejavu" in text # the watermark filter font
219         assert "torch==1.11.0+cu113" in text # the verified WASB stack
220         assert "WASB-SBDT" in text # the shuttle detector repo
221         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
222         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
223

```

12. user (2026-06-21T10:45:31.796Z)

```

1     # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2     #
3     # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4     # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5     # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6     # dispatch shim is proven with a fake client (tests/test_compute_backend.py).

```

```

7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40         torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45     # --- Main application env ---
46     WORKDIR /app
47     COPY pyproject.toml README.md ./
48     COPY backend ./backend
49     COPY training ./training
50     RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54     ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55         WASB_REPO_DIR=/opt/models/WASB-SBDT \

```

```

56     WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57     RALLY_DETECTOR_IMPL=native
58
59     # The dispatcher (CloudRunCompute) supplies the per-execution args:
60     #   infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61     ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

13. user (2026-06-21T10:45:32.268Z)

```

1     # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3     The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4     dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5     a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7     > Status: the dispatch shim + image are landed + mock-tested (M6). The real
build + push +
8     > run on an L4 is the M7 owner step, within the $100/?10k budget (D17) ?
confirm cmd + hourly
9     > cost first, delete every resource after ([gcp-vm-always-delete]). CI does
not build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run Job (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29         --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30         --max-retries 0 --task-timeout 3600 \
31         --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```
34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the cloud-serving profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38     ```bash
39     export DEPLOYMENT_PROFILE=cloud-serving
40     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT

```

```

41      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44      ```
45
46      ## 4. M7 acceptance (the owner runlog)
47      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52      ## Gotchas
53      - The dispatched container must run inline ? `CloudRunCompute` forces
`compute_target=local_gpu`
54      + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a bucket marker (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56      ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A failed Job currently writes no marker ? the dispatcher's bounded poll times
out
58      (`ExecutionPolicy.max_wait_sec`). A failure-marker is a follow-up.
59      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
60

```

14. assistant (2026-06-21T10:45:36.221Z)

Now let me verify the critical pieces around the lens. First, the `--set` override ordering (later wins?), the `load\_config` / deployment-profile interaction, and whether the `cloud-serving` profile forces `compute\_target=cloud\_run` in the container (which would cause recursion).

15. user (2026-06-21T10:45:37.313Z)

```

deploy\cloudrun\README.md:37:`compute_target=cloud_run`). `get_compute_backend(config)` then
builds `CloudRunCompute` from env:
deploy\cloudrun\README.md:42:# running in-process; the container runs the SAME worker INLINE
(compute_target forced local_gpu).
deploy\cloudrun\README.md:53:- The dispatched container must run inline ? `CloudRunCompute`
forces `compute_target=local_gpu`
backend\worker\_main_.py:119:    container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend\worker\_main_.py:152:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend\worker\_main_.py:155:    # dispatcher forces compute_target=local_gpu), so it never
recursively re-dispatches.
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
`deployment.{profile,compute_target}` plus
backend\config\presets.py:45:    "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:    "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:58:    "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},

```

```

backend\config\models.py:351: compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\loader.py:167: ct = merged["deployment"].get("compute_target")
backend\config\loader.py:170: # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:172: # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend\config\loader.py:173: merged["deployment"]["compute_target"] = canonical
backend\config\loader.py:176: "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:180: logger.info("[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:181: profile,
merged["deployment"].get("compute_target"))
docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_truly-local_TEMPLATE.md:6:> **Profile:** `truly-
local` (`compute_target=local_gpu`, `storage=local`, stitch in-process, zero cloud).
backend\compute\__init__.py:4:`ComputeBackend` ONLY when a non-local
`deployment.compute_target` is selected, and `None`
backend\compute\__init__.py:39: """Return the compute backend for
`config.deployment.compute_target`, or `None` to run
backend\compute\__init__.py:42: Only `compute_target == "cloud_run"` returns a non-None
backend (a `CloudRunCompute`). Its
backend\compute\__init__.py:47: target = getattr(getattr(config, "deployment", None),
"compute_target", "") or ""
backend\compute\cloud_run.py:14:The dispatched container is forced to run **INLINE**
(`compute_target=local_gpu` + native detector)
backend\compute\cloud_run.py:38:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend\compute\cloud_run.py:40: "deployment.compute_target=local_gpu",
docs\COMPUTE_DECOUPLED_SERVING\README.md:29:| D2 | **Two registries:** `ComputeBackend` (where)
selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by
`input_backend`/`storage.backend`. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's
`local`/`hosted`/`byo_worker`. |
docs\COMPUTE_DECOUPLED_SERVING\README.md:77:| **truly-local** | `compute_target=local_gpu`;
`detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` =
mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker |
Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
docs\COMPUTE_DECOUPLED_SERVING\README.md:78:| **cloud-serving** | `compute_target=cloud_run`;
`detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or
gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control
plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-
plane job; scale-to-zero | Hosted SaaS ? owner (a) |
docs\COMPUTE_DECOUPLED_SERVING\README.md:79:| **cpu-mock** | `compute_target=cpu_mock`;
`detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest |
CI, regression, profile-parity, GPU-free dev |
docs\COMPUTE_DECOUPLED_SERVING\README.md:85:- Selecting a profile applies a **coherent preset
bundle** of *existing* knobs that today drift independently: **INPUT** (new
`input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new
`compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy),
**STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
docs\COMPUTE_DECOUPLED_SERVING\README.md:116:| M1 | `deployment.profile` + `compute_target`
config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile=''
default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/279) |
docs\COMPUTE_DECOUPLED_SERVING\README.md:160:- 2026-06-21 ? **M0 APPROVED** ; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
backend\compute\base.py:8:The seam is **default-OFF**: only `deployment.compute_target ==
"cloud_run"` selects a
tests\test_config_deployment.py:6:- Each profile applies its coherent preset bundle
(compute_target + the existing
tests\test_config_deployment.py:50: assert cfg.deployment.compute_target == ""

```



```

tests\test_config_deployment.py:68:    assert dumped["deployment"] == {"profile": "",
"compute_target": ""}
tests\test_config_deployment.py:83:def test_compute_target_literal_matches_presets():
tests\test_config_deployment.py:86:    literal_targets =
set(typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation))
tests\test_config_deployment.py:110:    assert cfg.deployment.compute_target == "cpu_mock"
tests\test_config_deployment.py:119:    assert cfg.deployment.compute_target == "cloud_run"
tests\test_config_deployment.py:128:    assert cfg.deployment.compute_target == "local_gpu"
tests\test_config_deployment.py:163:def
test_config_json_can_override_compute_target_with_warning(tmp_path, caplog):
tests\test_config_deployment.py:165:    "deployment": {"profile": "cpu-mock",
"compute_target": "cloud_run"},
tests\test_config_deployment.py:170:    assert cfg.deployment.compute_target == "cloud_run" #
explicit override honored (D15)
tests\test_config_deployment.py:174:def
test_empty_compute_target_under_profile_backfills_to_canonical(tmp_path):
tests\test_config_deployment.py:175:    """An active profile whose compute_target is blanked to
"" in config.json gets its canonical
tests\test_config_deployment.py:176:    target back ? the record never silently records an
active profile with no compute_target."""
tests\test_config_deployment.py:178:    "deployment": {"profile": "cloud-serving",
"compute_target": ""},
tests\test_config_deployment.py:181:    assert cfg.deployment.compute_target == "cloud_run"
tests\test_config_deployment.py:200:    assert cfg.deployment.compute_target == "cpu_mock" #
cpu-mock preset applied
tests\test_config_deployment.py:228:    assert cfg.deployment.compute_target == "cpu_mock" #
preset fully applied
tests\test_config_deployment.py:252:    assert cfg.deployment.compute_target == ""
tests\test_config_deployment.py:358:    assert saved["deployment"] == {"profile": "",
"compute_target": ""}
docs\archives\decisions\DECISIONS.md:696:2. A **`ComputeBackend` / deployment profile** decides
**WHERE** the core runs (local process vs a Cloud Run GPU job); a **`StorageBackend`** decides
**WHERE I/O comes from** (local FS / USB / mounted Google Drive / bucket). **Both are selected
by ONE startup/setup config** (`deployment.profile` + `compute_target`), realizing the
PLATFORM_ARCHITECTURE `local|hosted|byo_worker` registry.
docs\archives\decisions\DECISIONS.md:712:- **Cost/new surface:** a new
`deployment`+`compute_target` config surface, a Cloud Run **dispatch shim**, a **containerized
GPU+ffmpeg worker image**, a **cost-ledger migration**, and **edge auth** must all be built +
maintained. Cross-GPU detection is **not** byte-identical ? parity asserted at the window level.
The `byo_worker` tier carries a real ops/support burden (deferred, power-user only).
tests\test_compute_backend.py:5:read flow is proven for $0. The cardinal invariant: **default-
OFF** ? only ``compute_target=cloud_run``
tests\test_compute_backend.py:46:def _cfg(compute_target="", storage_backend="local"):
tests\test_compute_backend.py:48:    "deployment": {"compute_target": compute_target},
tests\test_compute_backend.py:85:    """The dispatched container must run INLINE (never re-
dispatch) ? compute_target=local_gpu +
tests\test_compute_backend.py:99:    assert "deployment.compute_target=local_gpu" in joined
tests\test_compute_backend.py:103:    assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")
tests\test_compute_backend.py:120:    """cmd_infer with compute_target=cloud_run hands off to
the ComputeBackend (no in-process run)."""
tests\test_profile_parity.py:80:    banned = {"profile", "compute_target", "deployment",
"storage", "backend"}
tests\test_profile_parity.py:89:    The profiles differ only in compute_target/label, which the
pure core never reads."""

```

16. user (2026-06-21T10:45:38.879Z)

```

backend\config\_init__.py:19:from .presets import PROFILE_PRESETS, probe_cuda_available,
suggest_profile
backend\config\_init__.py:38:    "suggest_profile",
backend\config\presets.py:1:"""Deployment-profile presets + environment auto-detection
(COMPUTE_DECOUPLED_SERVING M1).
backend\config\presets.py:3:A *deployment profile* is a single switch that applies a coherent
BUNDLE of existing

```

```

backend\config\presets.py:4:config knobs, so the truly-local / cloud-serving / cpu-mock modes
don't drift apart one
backend\config\presets.py:9:    built-in model defaults < profile preset < config.json <
env < worker --set
backend\config\presets.py:12:module load**. The loader applies a preset ONLY when a profile is
EXPLICITLY selected
backend\config\presets.py:13:(``config.json`` ``deployment.profile`` or the
``DEPLOYMENT_PROFILE`` env var); with no
backend\config\presets.py:14:profile selected the loader is byte-identical to the pre-M1
behaviour (``profile=""`` ==
backend\config\presets.py:15:today). Auto-detect only ever **suggests** a profile ? it never
auto-applies one (D15: cloud
backend\config\presets.py:27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-
mock")
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:34:    "cloud-serving": "cloud_run",
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend\config\presets.py:45:    "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:51:    "cloud-serving": {
backend\config\presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:58:        "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\presets.py:59:        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The
CI / regression profile.
backend\config\presets.py:66:def is_known_profile(profile: str) -> bool:
backend\config\presets.py:67:    """True for a non-empty, recognised profile name."""
backend\config\presets.py:68:    return bool(profile) and profile in PROFILE_PRESETS
backend\config\presets.py:71:def preset_for(profile: str) -> dict[str, Any]:
backend\config\presets.py:72:    """Return a deep COPY of the preset bundle for ``profile`` (so
callers can't mutate the
backend\config\presets.py:73:    module-level table), or ``{}`` for ``""`` / an unknown
profile."""
backend\config\presets.py:74:    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
backend\config\presets.py:100:    Exposed for the per-profile startup checks
(``backend/config/startup.py``) so a caller
backend\config\presets.py:109:def suggest_profile(
backend\config\presets.py:112:    """SUGGEST (never auto-apply ? D15) a deployment profile from
the environment.
backend\config\presets.py:116:    - ``K_SERVICE`` set (Cloud Run)          ? ``"cloud-serving"``
backend\config\presets.py:122:    imports torch; a CLI ``--detect`` or cloud-serving startup
that wants the GPU probe passes
backend\config\presets.py:127:        return "cloud-serving"
backend\config\models.py:36:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
backend\config\models.py:63:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
backend\config\models.py:335:    """Deployment profile ? one switch that selects a coherent
bundle of compute + storage
backend\config\models.py:338:    **Pure / additive:** the empty default profile (``""``) applies
NO preset, so the pipeline
backend\config\models.py:339:    behaves exactly as before this section existed. A profile is
opt-in (``config.json``
backend\config\models.py:340:    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var);
auto-detect only ever
backend\config\models.py:342:    these fields yet (M1): selecting a profile works purely by pre-
setting the EXISTING knobs
backend\config\models.py:347:    # "" = no profile (today's manual knobs). Others apply
presets.PROFILE_PRESETS.
backend\config\models.py:348:    profile: Literal["", "truly-local", "cloud-serving", "cpu-
mock"] = ""
backend\config\models.py:349:    # Where the core runs. Set by the profile preset; overridable

```

```

in config.json. Inert in M1
backend\config\startup.py:1: """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
backend\config\startup.py:3: A deployment profile (`truly-local` / `cloud-serving` / `cpu-
mock`) implies a coherent
backend\config\startup.py:5: (expensive) job runs ? when the active profile is inconsistent with
the environment:
backend\config\startup.py:7: * **cloud-serving with a non-cloud storage backend** ? a hard
**error** (the one config bug we
backend\config\startup.py:8: can detect with certainty + zero false-positive risk: a cloud
profile that can't reach a
backend\config\startup.py:18: **DEFAULT-OFF:** with no profile selected (`profile == ""`) this
is a strict **no-op** ? exactly
backend\config\startup.py:19: the pre-M5 behaviour. Only an explicitly-selected profile triggers
any check (mirrors the loader,
backend\config\startup.py:20: which applies a preset only for an explicit profile). So nothing
changes for today's manual-knob
backend\config\startup.py:34: # Storage backends that count as "a cloud bucket" for cloud-serving
coherence.
backend\config\startup.py:45: """One per-profile finding. `level` is `error` (fatal) or
`warn` (heads-up)."""
backend\config\startup.py:53: """Raised by `enforce_startup_checks` when an active profile
fails a FATAL check."""
backend\config\startup.py:58:         "deployment profile startup check failed: "
backend\config\startup.py:75: """Return the per-profile findings for `config` (empty when
no profile is selected).
backend\config\startup.py:81: profile = getattr(getattr(config, "deployment", None),
"profile", "") or ""
backend\config\startup.py:82: if not profile:
backend\config\startup.py:83:     return [] # DEFAULT-OFF: no profile ? no checks (identical
to pre-M5).
backend\config\startup.py:93: if profile == "truly-local":
backend\config\startup.py:99:     f"profile 'truly-local' usually uses local/gdrive
storage, but storage.backend="
backend\config\startup.py:120: elif profile == "cloud-serving":
backend\config\startup.py:121:     # The ONE fatal coherence check: a cloud profile that
can't reach a bucket can't serve.
backend\config\startup.py:125:         f"profile 'cloud-serving' requires a cloud storage
backend "
backend\config\startup.py:138:         "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
backend\config\startup.py:144:         "profile 'cloud-serving' expects a GPU but no CUDA
GPU is visible to this process.",
backend\config\startup.py:147: elif profile == "cpu-mock":
backend\config\startup.py:148:     # The mock profile is for CI/regression ? a real detector
here defeats the purpose.
backend\config\startup.py:152:         f"profile 'cpu-mock' normally uses
detector_impl='stub', but detector_impl="
backend\config\startup.py:171: and no raise when no profile is selected
(`run_startup_checks` returns `[]`)."""
backend\config\loader.py:23: is_known_profile,
backend\config\loader.py:25: suggest_profile,
backend\config\loader.py:35: replace). Used ONLY for the profile-preset precedence path so a
config.json sub-key
backend\config\loader.py:36: overrides a preset sub-key while sibling preset/default sub-keys
survive ? the no-profile
backend\config\loader.py:47: def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
backend\config\loader.py:48: """The EXPLICITLY chosen deployment profile, env winning over
config.json. ```` if none
backend\config\loader.py:50: environment never silently selects a profile (and never silently
spends on cloud).
backend\config\loader.py:52: An *unrecognised* `DEPLOYMENT_PROFILE` env value warns and
FALLS THROUGH to the file's
backend\config\loader.py:53: profile (rather than suppressing it) ? so an env typo can't
leave the recorded profile
backend\config\loader.py:56: env_profile = os.environ.get("DEPLOYMENT_PROFILE")

```

[Showing results with pagination = limit: 80]

17. assistant (2026-06-21T10:45:45.814Z)

This is a critical area. The container runs with `RALLY\_DETECTOR\_IMPL=native` and the dispatcher passes `--set deployment.compute\_target=local\_gpu`. But the container also has `DEPLOYMENT\_PROFILE` env potentially set. Let me check the precedence in `loader.py` ? specifically whether the `cloud-serving` profile preset could re-set `compute\_target=cloud\_run` AFTER (or in a way that overrides) the `--set` override in the container. This is the recursion concern (lens item 3).

Key: in the container, `--set deployment.compute\_target=local\_gpu` is applied by `\_apply\_overrides` AFTER `load\_config`. But does `load\_config` itself set `compute\_target=cloud\_run` from the profile, and then `\_apply\_overrides` flips it back to `local\_gpu`? Let me read the loader precedence and confirm `\_apply\_overrides` order vs the profile, and confirm the dispatch branch is after `\_apply\_overrides`.

18. user (2026-06-21T10:45:46.812Z)

```
140         cfg_path, ", ".join(sorted(unknown)),
141     )
142
143     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
144     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
145     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
146     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
147     defaults = _get_builtin_defaults()
148     profile = _resolve_explicit_profile(file_data)
149
150     if profile and not is_known_profile(profile):
151         # Only an unrecognised value from config.json reaches here (env typos already
fell back
152         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
153         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
154         logger.warning(
155             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
156             profile, ", ".join(PROFILE_PRESETS),
157         )
158         profile = ""
159
160     if profile:
161         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
162         # Record the profile actually applied ? the env may have chosen it over
config.json,
163         # so re-stamp it onto the merged deployment section to keep the record honest.
164         merged.setdefault("deployment", {})
165         if isinstance(merged["deployment"], dict):
166             merged["deployment"]["profile"] = profile
167             ct = merged["deployment"].get("compute_target")
168             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
169             if not ct and canonical:
170                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
171                 # to "") gets its canonical target back ? the record never silently lies
about an
172                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
173                 merged["deployment"]["compute_target"] = canonical
```

```

174         elif ct and canonical and ct != canonical:
175             logger.warning(
176                 "[CONFIG] deployment profile %r normally uses compute_target=%r but
177                 "
178                 "config.json overrides it to %r ? honoring the explicit override.",
179                 profile, canonical, ct,
180             )
181             logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
182                         profile, merged["deployment"].get("compute_target"))
183         else:
184             # No profile ? the original shallow merge (Pydantic re-defaults nested
185             sections).
186             merged = {**defaults, **file_data}
187             suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
188             (D15)
189             if suggestion:
190                 logger.debug(
191                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
192                     %r ? "
193                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
194                     never "
195                     "auto-applies ? D15).", suggestion,
196                 )
197             return AppConfig.model_validate(merged)
198
199     def save_default_config(path: str | Path | None = None) -> Path:
200         """Write a fresh default config.json.
201
202         Used by scripts/doctor.py diagnostics and the --setup CLI path.

```

19. assistant (2026-06-21T10:45:55.370Z)

This confirms the critical recursion analysis. In the container:

- The Dockerfile sets `RALLY_DETECTOR_IMPL=native` but does NOT set `DEPLOYMENT_PROFILE`.
- The README's deploy step (`gcloud run jobs create`) does NOT set `DEPLOYMENT_PROFILE` either ? only `WASB_WEIGHTS`.
- So in the container, `load_config()` resolves `profile=""` (no profile env, no config.json with profile baked into the image). Then `_apply_overrides` applies `--set deployment.compute_target=local_gpu`. So `compute_target=local_gpu` ? `get_compute_backend` returns None ? inline. Good, no recursion in the documented path.

But there's a real concern: the recursion safety depends entirely on `DEPLOYMENT_PROFILE` NOT being set to `cloud-serving` in the container. Let me verify whether `_resolve_explicit_profile` (env) would override the `--set` and re-introduce `cloud_run`. The key question: does `--set deployment.compute_target=local_gpu` (applied last by `_apply_overrides`) win over the profile preset applied inside `load_config`?

Reading the loader: the profile preset sets `merged["deployment"]["compute_target"]`, but then `_apply_overrides(config, args.set)` runs AFTER `load_config` returns and does `setattr(obj, "compute_target", "local_gpu")` ? so `--set` wins. Good. Let me verify `_apply_overrides` ordering (later wins) and confirm the dispatch branch is after it, plus check the README's profile env interaction once more.

20. user (2026-06-21T10:45:56.666Z)

```

47     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
48         """The EXPLICITLY chosen deployment profile, env winning over config.json. ```` if
49         none
50         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
51         a bare
52         environment never silently selects a profile (and never silently spends on cloud).

```

```

52         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
53         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
54         asserting a preset that was never applied. A bad value in config.json is left to
surface as
55         a hard, typed-Literal error downstream (config-file correctness)."""
56         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
57         if env_profile and env_profile.strip():
58             ep = env_profile.strip()
59             if is_known_profile(ep):
60                 return ep
61             logger.warning(
62                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
63                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
64             )
65             # fall through to the file's profile (env typo must not suppress a valid file
choice)
66         dep = file_data.get("deployment")
67         if isinstance(dep, dict):
68             prof = dep.get("profile")
69             if isinstance(prof, str) and prof.strip():
70                 return prof.strip()
71         return ""
72
73
74     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
75         """Dotted paths in `data` that the typed config will not honor.
76

```

21. assistant (2026-06-21T10:46:09.751Z)

Now I have the full picture for the recursion analysis. Let me reason through the scenarios:

****Scenario A (documented happy path):**** Container has `RALLY\_DETECTOR\_IMPL=native`, no `DEPLOYMENT\_PROFILE`, no profile in config.json. `load\_config` ? `profile=""`, `compute\_target=""`. `\_apply\_overrides` applies `--set deployment.compute\_target=local\_gpu` ? `local\_gpu`. `get\_compute\_backend` returns None ? inline. ? Safe.

****Scenario B (the recursion risk):**** If the container were run with `DEPLOYMENT\_PROFILE=cloud-serving` set (e.g., owner sets it on the Job following the README §3 which exports it on the CONTROL PLANE ? but the control plane and container are separate). In the container, `load\_config` would set `compute\_target=cloud\_run`. Then `\_apply\_overrides` applies `--set deployment.compute\_target=local\_gpu` which runs AFTER and wins ? `local\_gpu`. So even with `DEPLOYMENT\_PROFILE=cloud-serving` set in the container, the `--set` override flips it back. ? Safe ? because `--set` is applied after `load\_config`.

Let me confirm `\_apply\_overrides` truly applies in-order (later wins) when the SAME key appears twice, which is the test's claim (user `sport=tennis` then inline overrides). The loop does `setattr` per kv in order, so the last `deployment.compute\_target=X` wins. The dispatcher appends `\_DEFAULT\_CONTAINER\_OVERRIDES` AFTER `spec.config\_overrides`, so if a user passed `--set deployment.compute\_target=cloud\_run`, the inline `deployment.compute\_target=local\_gpu` comes later and wins. ? This is exactly lens item 3, and it's handled correctly.

Now let me verify lens item 5 (dispatch branch is after `\_apply\_overrides` and startup checks) ? confirmed in cmd_infer: line 147 `load\_config`, 148 `\_apply\_overrides`, 149 startup checks, 156 `get\_compute\_backend`. ?

Let me now run the test suite for this module and a quick lint to verify it's grounded, and double-check one subtle thing: the `get\_compute\_backend` call in `cmd\_infer` ignores `args.set` re-introduction at the DISPATCHER level. Let me verify there's no path where a user's `--set deployment.compute\_target=cloud\_run` at the dispatcher gets carried into config_overrides AND

wins over the inline override in the container.

22. user (2026-06-21T10:46:18.960Z)

```
..... [100%]  
9 passed in 6.36s
```

23. assistant (2026-06-21T10:46:31.067Z)

Tests pass. Now let me think harder about lens item 3, the most important re-dispatch concern. There's a subtle path I want to confirm.

At the **dispatcher** (control plane), `cmd_infer`` runs with `compute_target=cloud_run``. It builds `ComputeJobSpec`` with `config_overrides=tuple(args.set or ())``. If the user ran the dispatcher with `--set deployment.compute_target=cloud_run`` (redundant, or explicit), that gets passed into `config_overrides``. Then `CloudRunCompute.run_infer`` builds argv as `(*spec.config_overrides, *self._container_overrides)``, so the order is: user's `deployment.compute_target=cloud_run`` FIRST, then inline `deployment.compute_target=local_gpu`` LAST. In the container, `_apply_overrides`` applies in order ? `local_gpu`` wins. ? Safe.

But what about a user `--set deployment.compute_target=cloud_run`` that comes AFTER... no, the container overrides are always appended last by the dispatcher. There is no way for user `--set`` to come after the inline overrides. ?

Let me check one more edge: what if `container_overrides`` is explicitly injected as empty `()`` via the `CloudRunCompute`` constructor? That's a config option. Let me verify that's not reachable from `get_compute_backend`` (it isn't ? the factory never passes `container_overrides``, so it defaults to `_DEFAULT_CONTAINER_OVERRIDES``). ?

Now let me verify a real concern about lens item 4 (done-marker default-OFF) and check whether the marker is written only on the success path. Looking at `cmd_infer``: the `--done-uri`` marker is written at line 247-248, but ONLY after the success return path (line 249 `return 0``). All the FAILURE returns (rc 2, rc 3) happen BEFORE line 247. So a failed container writes NO marker. This matches the README's documented behavior ("A failed Job currently writes no marker"). The default-OFF claim (no `--done-uri`` = no marker) is verified by `test_worker_no_done_uri_is_unchanged``. ?

Let me now check one genuinely subtle thing about the `_dispatch_remote`` returncode and whether the dispatcher's `cmd_infer`` correctly avoids running the in-process path twice. Let me also verify the `config_overrides`` from the dispatcher includes the profile that selected cloud ? confirming the container won't ALSO get a profile that re-selects cloud via a baked config.json in the image.

24. user (2026-06-21T10:46:32.908Z)

```
Dockerfile  
README.md  
---check for baked config.json in image---  
10:# * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;  
11:# * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as a  
subprocess  
20:ENV DEBIAN_FRONTEND=noninteractive \  
26:# ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB env  
build.  
29:      python3 python3-pip python3-venv \  
32:# --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified torch-1.11  
stack) ---  
45:# --- Main application env ---  
47:COPY pyproject.toml README.md ./  
48:COPY backend ./backend  
49:COPY training ./training  
52:# The native runner finds the WASB env + repo by env (no `conda activate` needed); weights  
come in at  
54:ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \  

```

25. assistant (2026-06-21T10:46:43.502Z)

Confirmed: the Dockerfile does NOT bake a `config.json` and does NOT set `DEPLOYMENT\_PROFILE`. The container only sets `RALLY\_DETECTOR\_IMPL=native`. So in the container:

- No `config.json` ? `load\_config()` reads built-in defaults with `profile=""`, `compute\_target=""`.
- `--set deployment.compute\_target=local\_gpu` from the dispatcher ? `local\_gpu`.
- `get\_compute\_backend` returns None ? inline. ? No recursion.

Now, there's one genuinely subtle thing worth checking. If the owner runs the Cloud Run Job with `--set-env-vars DEPLOYMENT\_PROFILE=cloud-serving` (which README §3 instructs for the **control plane** but an owner could mistakenly apply to the Job too), `load\_config` sets `compute\_target=cloud\_run`, but `--set deployment.compute\_target=local\_gpu` (applied after) wins. So even that mistake is defended. This is robust because `--set` is always applied after `load\_config`. Good ? this is a strength, not a finding.

Let me now double-check one final correctness concern that's directly in scope: in `cmd\_infer`, the dispatch branch passes `get\_compute\_backend(config)` ? but note the config passed to the dispatcher already had `\_apply\_overrides` applied. If a user ran the DISPATCHER with `--set deployment.compute\_target=local\_gpu` to force-disable cloud, would the dispatch correctly NOT fire? Yes ? `\_apply\_overrides` flips config to `local\_gpu` before `get\_compute\_backend` is called (line 156), so it returns None ? inline. ? The dispatch decision honors `--set`. This confirms lens item 5.

Let me verify there are no other callers of `get\_compute\_backend` (e.g., the async job model / API) that might bypass the `\_apply\_overrides`-then-dispatch ordering and create a different recursion surface.

26. user (2026-06-21T10:46:44.702Z)

```
tests\test_compute_backend.py:11:from backend.compute import ComputeJobSpec, get_compute_backend
tests\test_compute_backend.py:55:     assert get_compute_backend(_cfg("")) is None
tests\test_compute_backend.py:56:     assert get_compute_backend(_cfg("local_gpu")) is None
tests\test_compute_backend.py:57:     assert get_compute_backend(_cfg("cpu_mock")) is None
tests\test_compute_backend.py:61:     backend = get_compute_backend(_cfg("cloud_run", "gcs"),
run_client=_FakeRunClient())
tests\test_compute_backend.py:72:     result = backend.run_infer(ComputeJobSpec(
tests\test_compute_backend.py:89:     backend.run_infer(ComputeJobSpec(
tests\test_compute_backend.py:110:     backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk")))
tests\test_compute_backend.py:111:     backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk")))
tests\test_compute_backend.py:127:         def run_infer(self, spec):
tests\test_compute_backend.py:131:     monkeypatch.setattr(wmain, "get_compute_backend", lambda
config: _FakeBackend())
backend\worker\__main__.py:28:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\__main__.py:116:def _dispatch_remote(compute: Any, args: argparse.Namespace) ->
int:
backend\worker\__main__.py:124:     result = compute.run_infer(spec)
backend\worker\__main__.py:153:     # DEFAULT-OFF: get_compute_backend returns None for
"""/local_gpu/cpu_mock ? fall through to the
backend\worker\__main__.py:156:     compute = get_compute_backend(config)
backend\worker\__main__.py:158:     return _dispatch_remote(compute, args)
backend\compute\__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. `get_compute_backend(config)` is
the seam: it returns a
backend\compute\__init__.py:19:     "ComputeBackend", "ComputeJobSpec", "ComputeResult",
"get_compute_backend",
backend\compute\__init__.py:32:def get_compute_backend(
backend\compute\cloud_run.py:101:     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
backend\compute\base.py:51:     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
```

27. assistant (2026-06-21T10:46:53.175Z)

`get\_compute\_backend` is called from exactly ONE production site (`cmd\_infer`, line 156), which is after `\_apply\_overrides` and startup checks. There is no other dispatch surface (the

API/async job path is not yet wired to this, consistent with the M6 scope). No alternative recursion surface exists.

Now let me reconsider lens item 3 one more time ? there IS a real subtlety I want to nail down. The ``_dispatch_remote`` passes ``config_overrides=tuple(args.set or ())`` to the spec. The dispatcher's ``args.set`` is the user-supplied ``--set`` list. So if the user ran the dispatcher with ``--set deployment.compute_target=cloud_run``, that gets forwarded to the container's argv as ``--set deployment.compute_target=cloud_run`` (user) THEN ``--set deployment.compute_target=local_gpu`` (inline). In the container, ``_apply_overrides`` applies both in order, last wins ? ``local_gpu``. ? Safe, and the test ``test_dispatch_argv_forces_inline_and_carries_overrides`` asserts exactly this ordering.

This is all robust. Let me do a final check on the ``_apply_overrides`` "later wins" guarantee for a duplicated key ? the loop iterates and ``setattr`s` each, so for two ``deployment.compute_target=`` entries the second overwrites the first. This is dict-attribute setattr, deterministic last-wins. ?

Let me run the full deployment config + worker tests to make sure nothing regressed and the contract holds end-to-end.

28. user (2026-06-21T10:47:02.856Z)

```
..... [100%]  
46 passed in 7.13s
```

29. assistant (2026-06-21T10:47:07.595Z)

All green. Let me do a final verification of one boundary I haven't explicitly confirmed: that the inline path (when ``get_compute_backend`` returns None) is truly byte-identical to pre-M6. The only additions to ``cmd_infer`` are: (1) the dispatch branch at lines 156-158 which is skipped when compute is None, and (2) the ``--done-uri`` marker write at lines 247-248 which is skipped when ``args.done_uri`` is None. Both are no-ops on the default path. Let me confirm via git diff that no other line in the existing inline body changed.

30. user (2026-06-21T10:47:08.962Z)

=== (lines above are DELETIONS; empty = pure additions) ===

31. assistant (2026-06-21T10:47:20.298Z)

Confirmed: the change to ``backend/worker/__main__.py`` is **purely additive** ? zero deletions. The existing inline path body is untouched. When ``compute_target`` ? ``{"", local_gpu, cpu_mock}``, ``get_compute_backend`` returns None and the in-process path runs byte-identically. When ``--done-uri`` is absent, no marker is written.

Let me do my final lens summary across all 5 items:

1. **Inline path unchanged for non-cloud targets:** ? ``get_compute_backend`` returns None for ``""/`local_gpu`/`cpu_mock`` (loader.py + ``__init__.py:48``), dispatch branch skipped, body is pure-additive (zero deletions). Verified by ``test_factory_is_none_for_local_targets``.
2. **Container overrides win:** ? Dispatcher appends ``_DEFAULT_CONTAINER_OVERRIDES`` last in argv (``cloud_run.py:109``); ``_apply_overrides`` applies ``--set`` in order with last-wins setattr (``__main__.py:60-68``); container's ``load_config`` (no profile/no baked config) + ``--set deployment.compute_target=local_gpu`` ? ``local_gpu`` ? ``get_compute_backend`` returns None ? inline. No re-dispatch.
3. **User ``--set`` / config.json re-introducing cloud_run cannot cause recursion:** ? Because ``--set`` is applied AFTER ``load_config``, and the inline-forcing override is appended AFTER user overrides, ``local_gpu`` always wins in the container. Even ``DEPLOYMENT_PROFILE=cloud-serving`` set on the Job is defeated. No baked config.json in the image.
4. **Done-marker default-OFF:** ? Written only when ``args.done_uri`` is set (``__main__.py:247``), and only on the success return. No ``--done-uri`` = no marker. Verified by

`test\_worker\_no\_done\_uri\_is\_unchanged`.

5. ****Dispatch sits after startup checks + `\_apply\_overrides`**** ? `cmd\_infer` order: `load\_config` (147) ? `\_apply\_overrides` (148) ? startup checks (149) ? `get\_compute\_backend` (156). So `--set`/profile is fully seen before the dispatch decision.

I found no DEFAULT-OFF break and no re-dispatch loop. The lens finds nothing real to report ? the safety invariants hold, are well-tested, and the change is pure-additive.

32. user (2026-06-21T10:47:20.952Z)

Structured output provided successfully

33. assistant (2026-06-21T10:47:36.404Z)

I reviewed the M6 change on `feat/serving-m6-cloudrun-dispatch` (HEAD `2266e91`) through the DEFAULT-OFF SAFETY & RE-DISPATCH CORRECTNESS lens. No real, grounded issues found ? empty findings.

Summary of what I verified (all confirmed safe):

1. ****Inline path unchanged for non-cloud targets**** ? `get\_compute\_backend` returns `None` for `"/local\_gpu"/cpu\_mock` (`backend/compute/\_\_init\_\_.py:48`), so the dispatch branch is skipped. The `cmd\_infer` change is ****pure-additive**** (git diff shows zero deletions in `backend/worker/\_\_main\_\_.py`); the existing in-process body is byte-identical. Covered by `test\_factory\_is\_none\_for\_local\_targets`.
2. ****Container overrides actually win**** ? `CloudRunCompute.run\_infer` appends `\_DEFAULT\_CONTAINER\_OVERRIDES` (`deployment.compute\_target=local\_gpu`, `indexing.detector\_impl=native`) ****last**** in argv (`backend/compute/cloud\_run.py:109`). `\_apply\_overrides` applies `--set` in order with last-wins `setattr` (`backend/worker/\_\_main\_\_.py:60-68`). In the container, `load\_config()` sees no profile and no baked config.json (Dockerfile sets only `RALLY\_DETECTOR\_IMPL=native`), so after `--set deployment.compute\_target=local\_gpu` the target is `local\_gpu` ? `get\_compute\_backend` returns `None` ? inline. No re-dispatch.
3. ****No recursion via user `--set` or container config**** ? because `--set` is applied after `load\_config`, and the inline-forcing overrides are appended after any user `config\_overrides`, `local\_gpu` always wins in the container even if a user passed `--set deployment.compute\_target=cloud\_run` at the dispatcher, or if `DEPLOYMENT\_PROFILE=cloud-serving` were mistakenly set on the Job (loader.py:160-181 sets `cloud\_run`, but the later `--set` flips it). Asserted by `test\_dispatch\_argv\_forces\_inline\_and\_carries\_overrides`.
4. ****Done-marker default-OFF**** ? written only when `args.done\_uri` is set, and only on the success return path (`backend/worker/\_\_main\_\_.py:247-248`); failure returns (rc 2/3) precede it, matching the documented "failed Job writes no marker." Covered by `test\_worker\_no\_done\_uri\_is\_unchanged`.
5. ****Dispatch sits after startup checks + `\_apply\_overrides`**** ? `cmd\_infer` order is `load\_config` (147) ? `\_apply\_overrides` (148) ? `\_run\_startup\_checks` (149) ? `get\_compute\_backend` (156), so `--set`/profile is fully resolved before the dispatch decision, and `--set deployment.compute\_target=local\_gpu` at the dispatcher correctly suppresses dispatch. `get\_compute\_backend` has exactly one production caller (`backend/worker/\_\_main\_\_.py:156`); no alternate dispatch surface exists. Ran `tests/test\_compute\_backend.py`, `tests/test\_config\_deployment.py`, `tests/test\_profile\_parity.py` ? 46 passed.